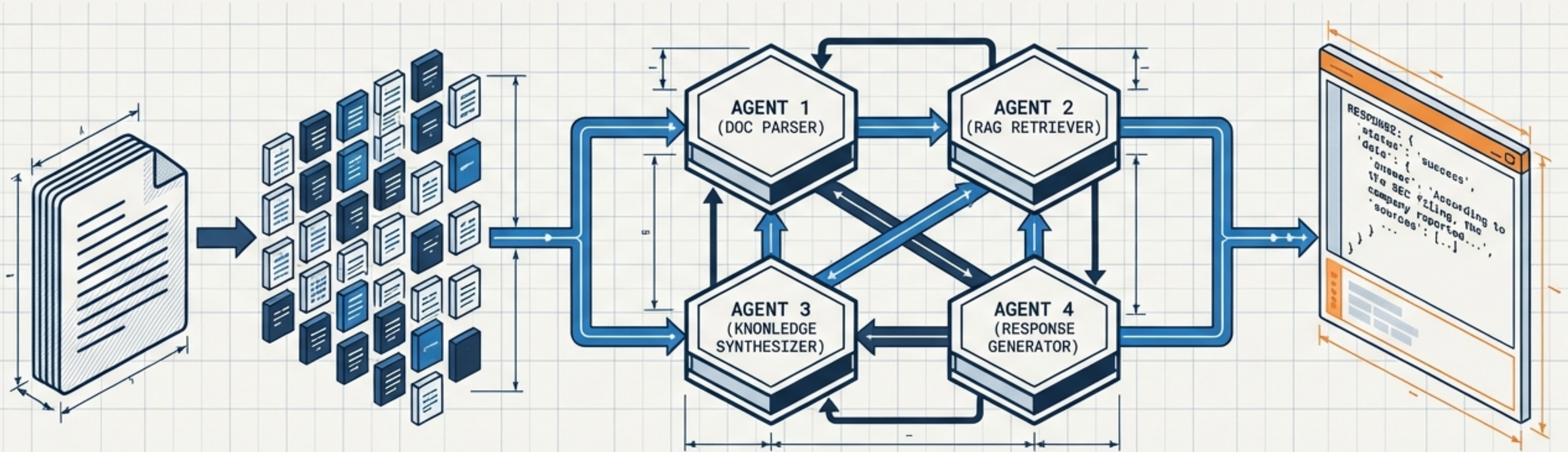


# FastAPI RAG Backend: Building an OpenAI-Compatible API with 8 Agent Architectures

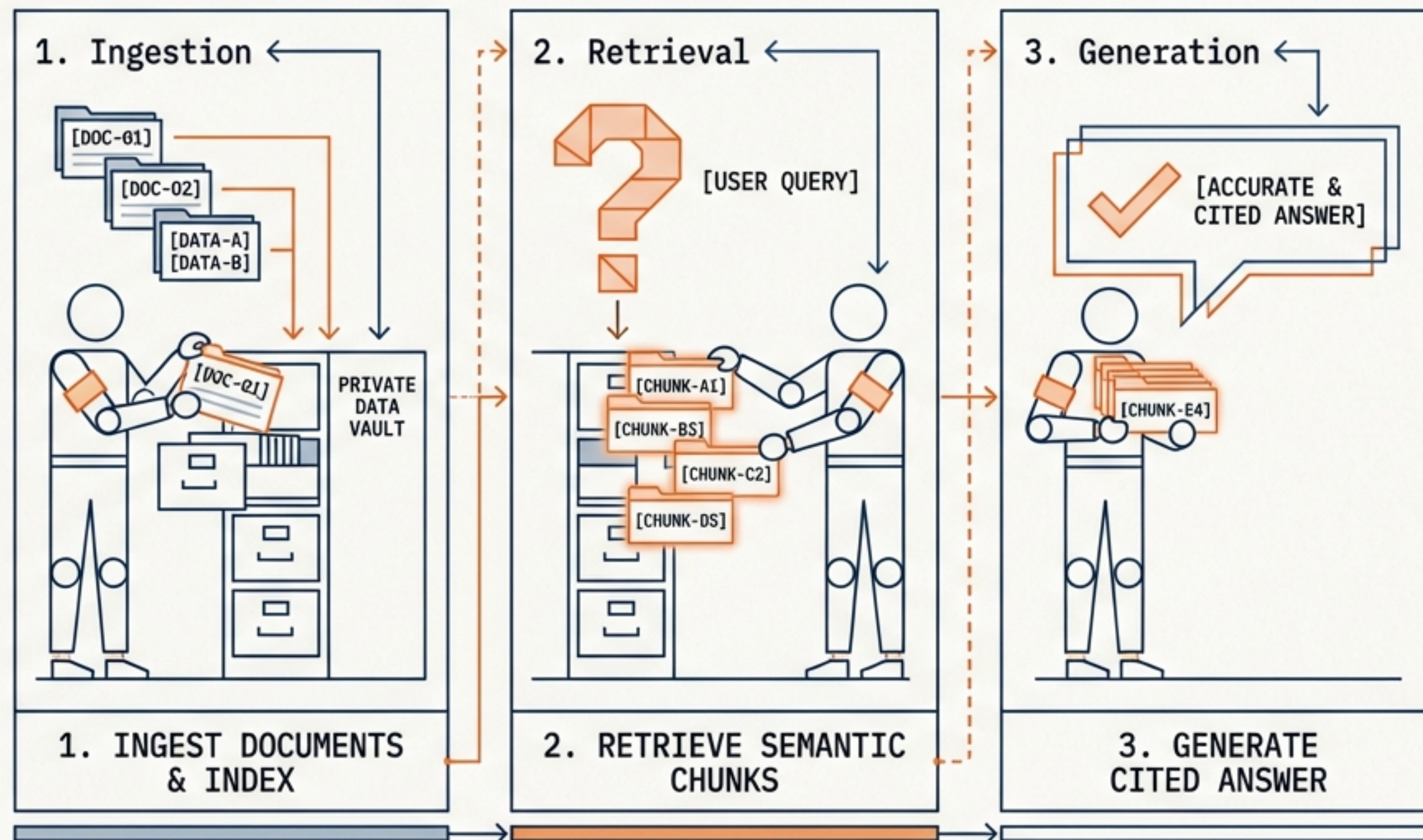
- From raw PDFs to autonomous multi-agent responses
- 3-Layer backend architecture for SEC financial filings and private corpora
- Progressive masterclass: LangChain, LangGraph, CrewAI, AutoGen, & MS Framework



# The RAG Paradigm: Grounding LLMs in Private Data

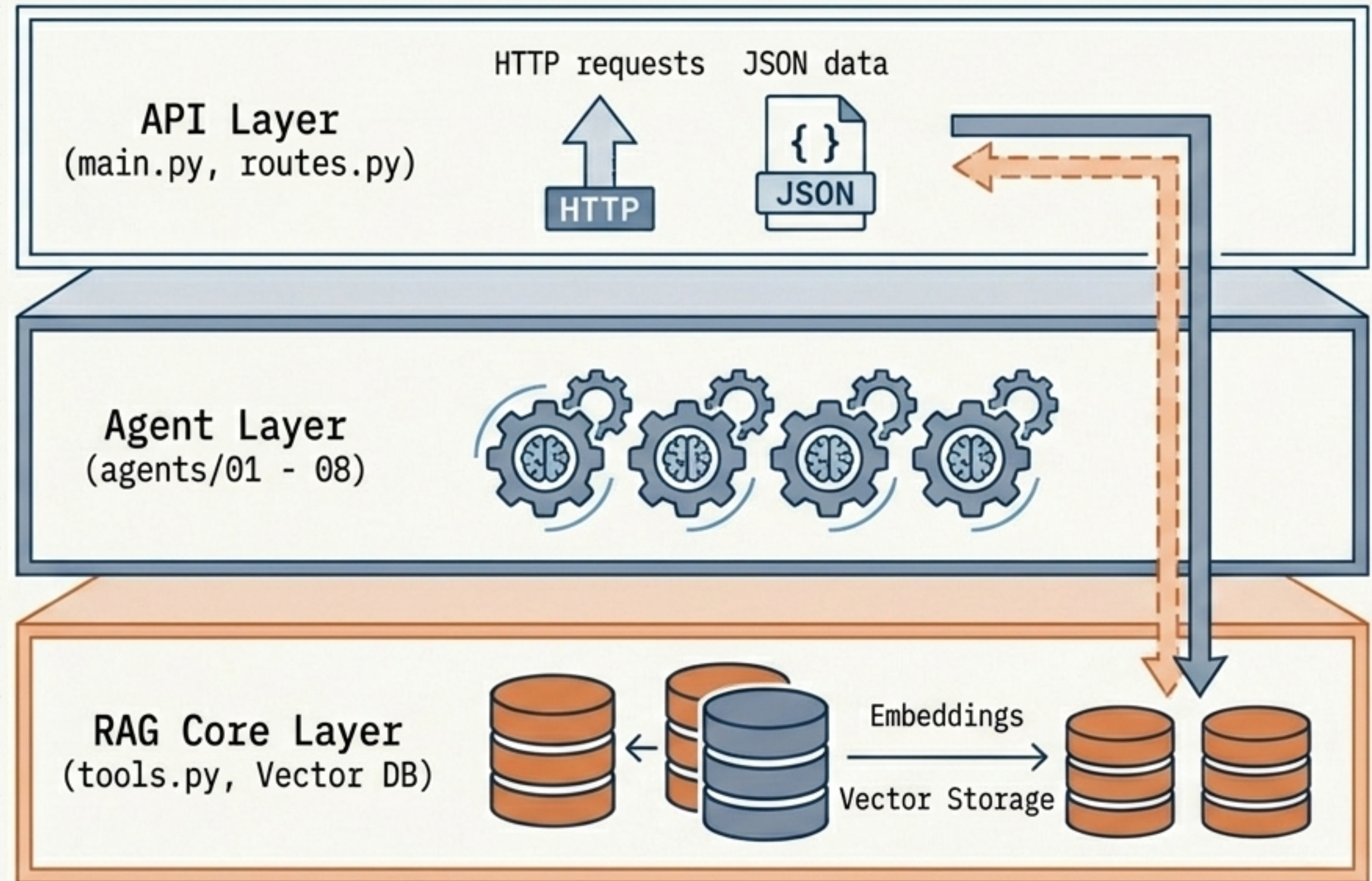
- **The Problem:** Base LLMs hallucinate and lack access to private, up-to-date company data.
- **The Solution:** Retrieval-Augmented Generation grounds models in factual, retrieved context.
- **The Process:** Ingest documents, retrieve semantic chunks, and generate a cited answer.

## Smart Librarian Analogy



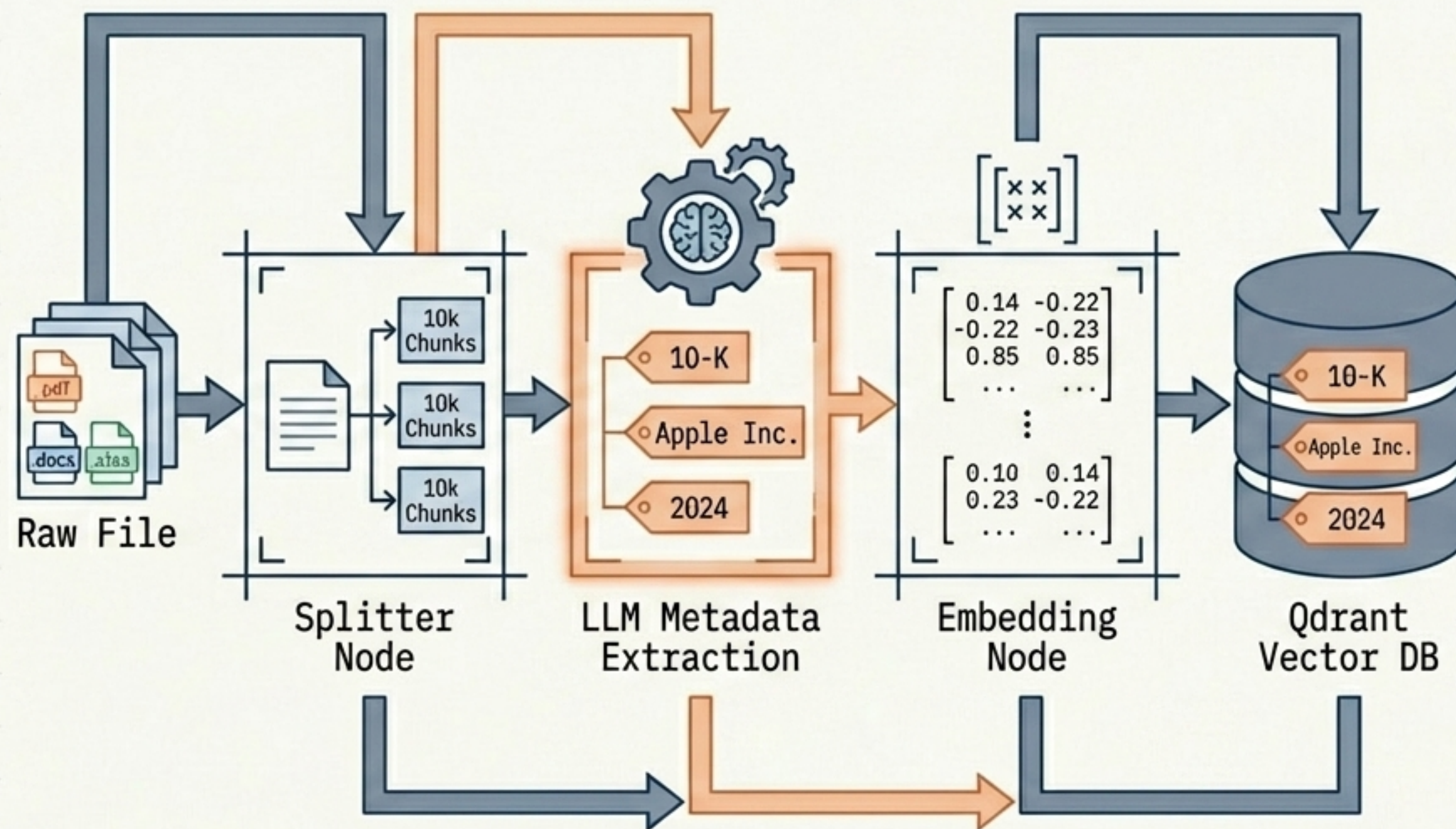
# The 3-Layer System Architecture

- **API Layer:** Handles HTTP endpoints, streaming, and OpenAI-compatible schemas.
- **Agent Layer:** Interchangeable reasoning engines executing logic across 8 architectures.
- **RAG Core Layer:** Persistent engine for embeddings, vector storage, and metadata.



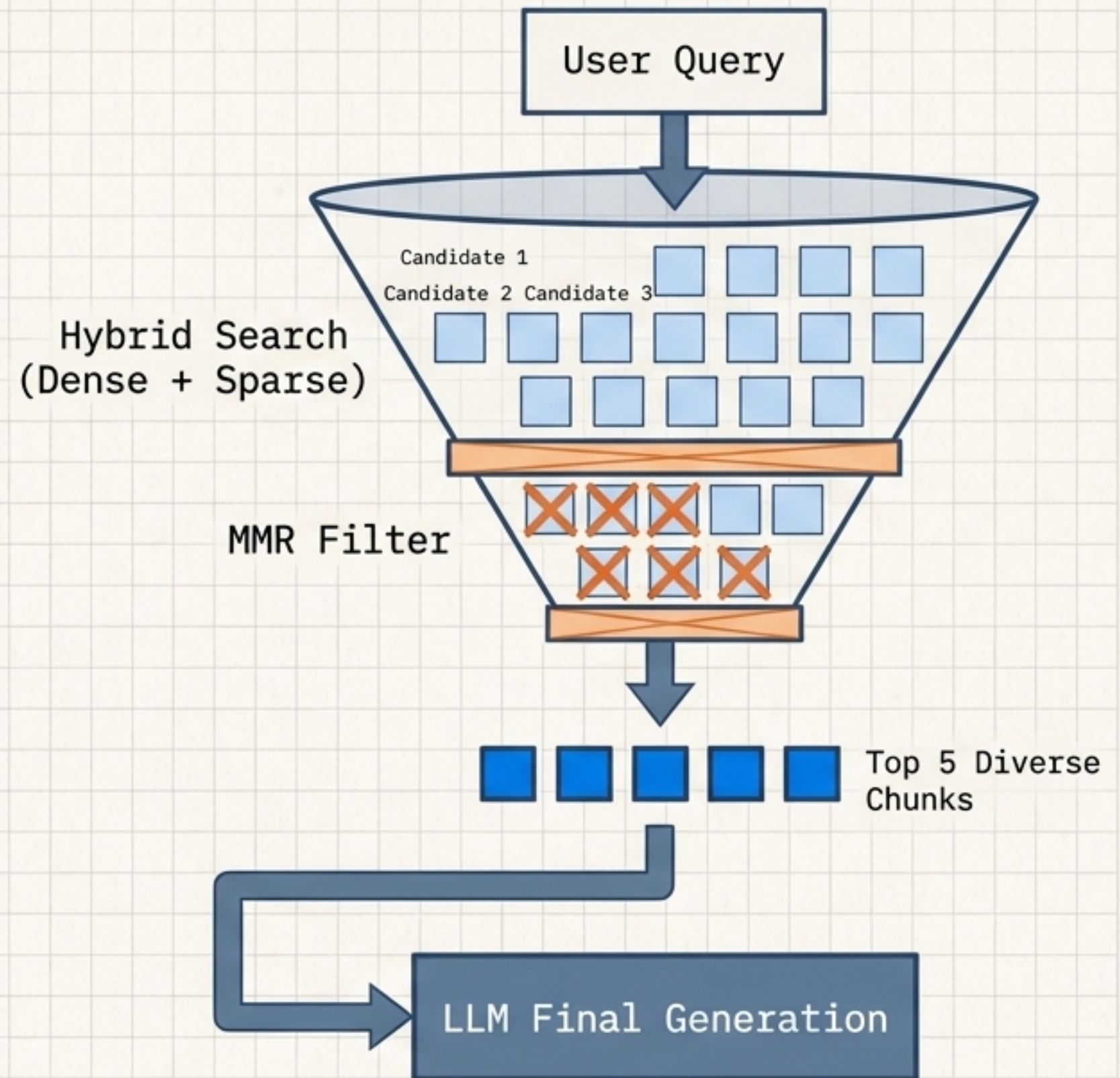
# Document Ingestion Pipeline: Making Files Searchable

- Accept complex documents (.pdf, .docx, .xlsx) via POST /upload.
- Chunk text (10,000 chars, 2,000 overlap) to preserve contextual continuity.
- LLM extracts metadata (Company, Doc Type, Year) at ingest time for hard filtering.
- Embed and store chunks + metadata into a Qdrant Vector Database.



# Query Processing: From Search to Answer

- **Context Discovery:** Agents query available metadata filters to narrow search scope.
- **Hybrid Search:** Combines dense semantic search with sparse BM25 keyword matching.
- **MMR Re-ranking:** Max Marginal Relevance balances relevance with diversity to avoid redundancy.
- **Synthesis:** Top 5 diverse chunks + Question are passed to the LLM for a cited response.



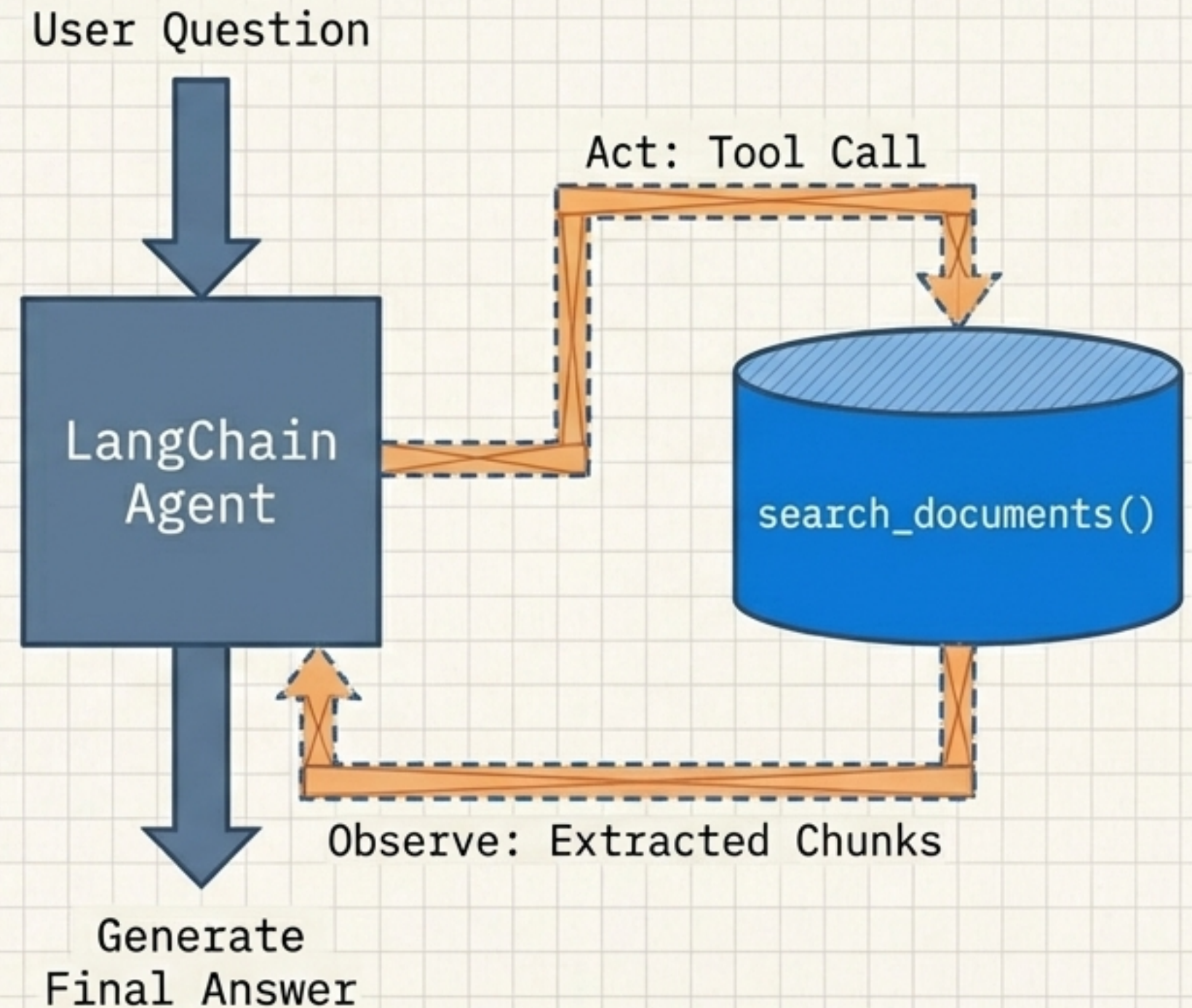
# OpenAI Compatibility & Config-Driven Core

- **Drop-in Replacement:** Endpoints strictly mirror OpenAI for seamless client integration.
- **Real-time Streaming:** Returns tokens via Server-Sent Events (SSE) exactly like ChatGPT.
- **Zero-Code Swaps:** YAML configs drive models, vector stores, and strategies dynamically.
- **Decoupled Secrets:** Dedicated .env files isolate credentials from application logic.



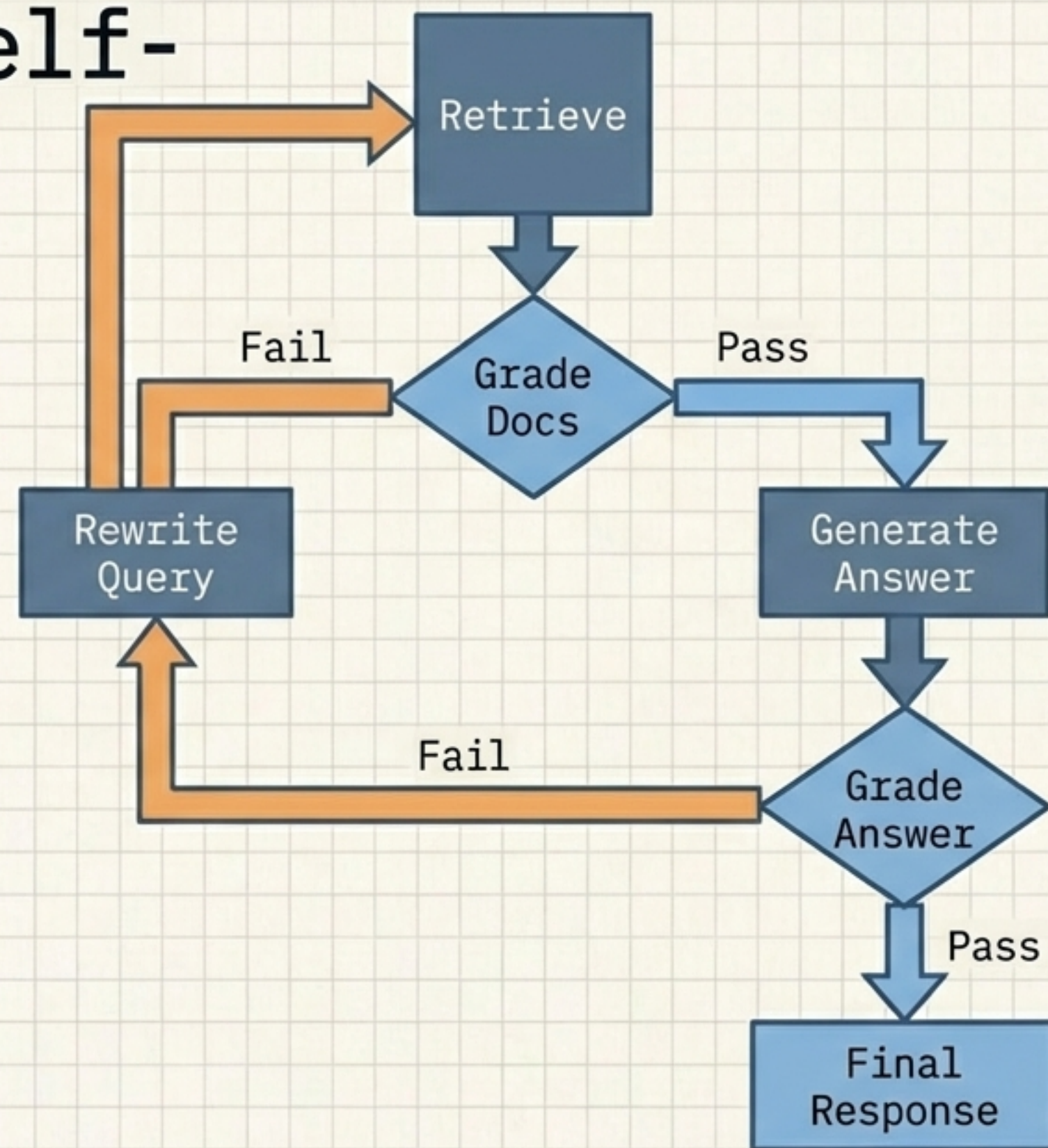
# Agent 01: The Baseline LangChain Agent

- **The Pattern:** A single autonomous agent equipped with vector search tools.
- **The Framework:** Built on the standard LangChain ReAct (Reason + Act) paradigm.
- **The Loop:** Reason (Need to search?) → Act (Call search tool) → Observe (Read chunks).
- **Execution:** Simplest architecture; streams response token-by-token directly to user.



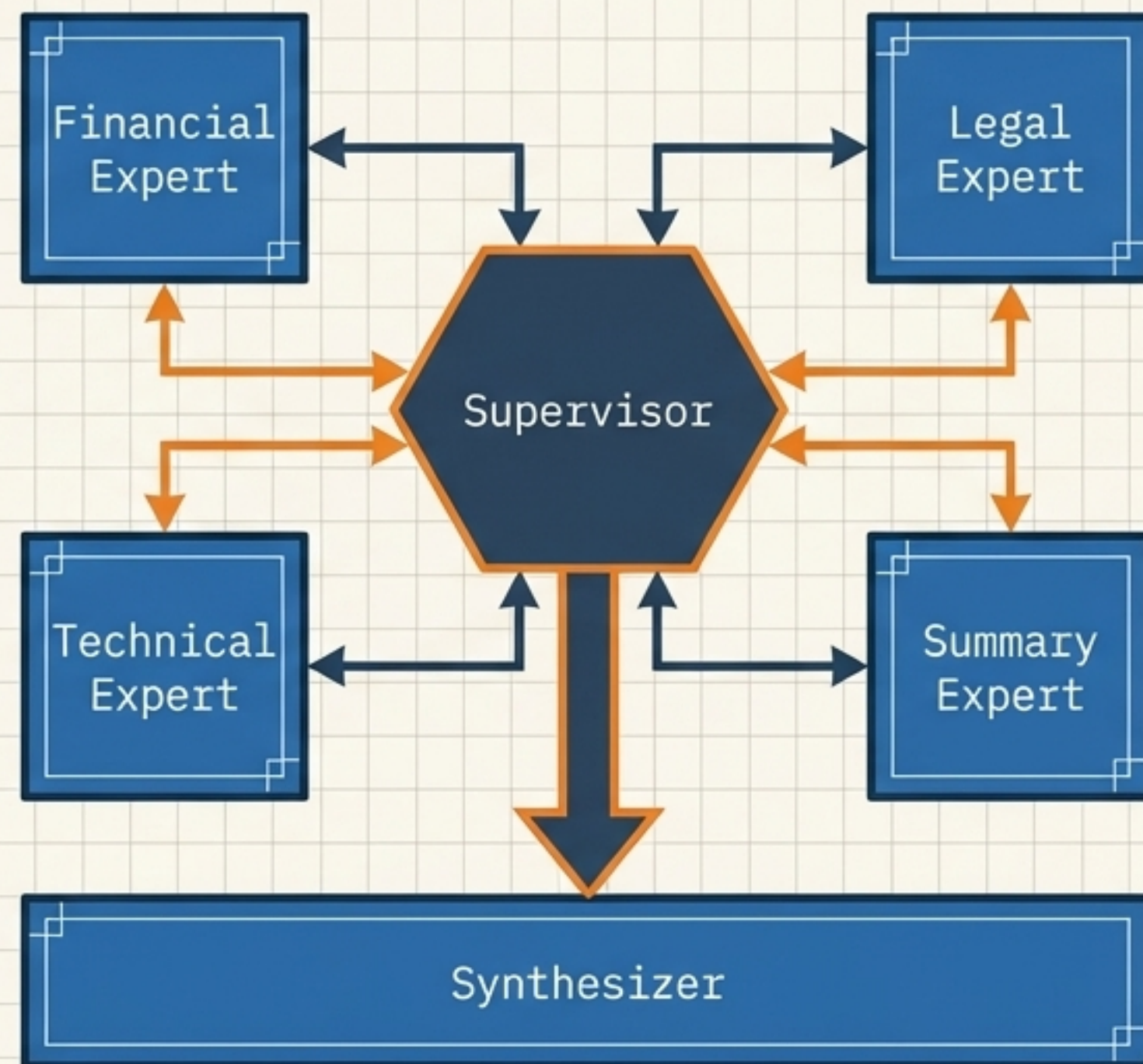
## Agent 02: LangGraph Self-Correcting Graph

- **The Pattern:** A state machine graph with built-in quality grading loops.
- **Document Grading:** Evaluates chunks. If irrelevant, rewrites query and searches again.
- **Answer Grading:** Evaluates answer against docs to aggressively prevent hallucinations.
- **Guardrails:** Enforces a maximum of 3 retry iterations to prevent infinite loops.



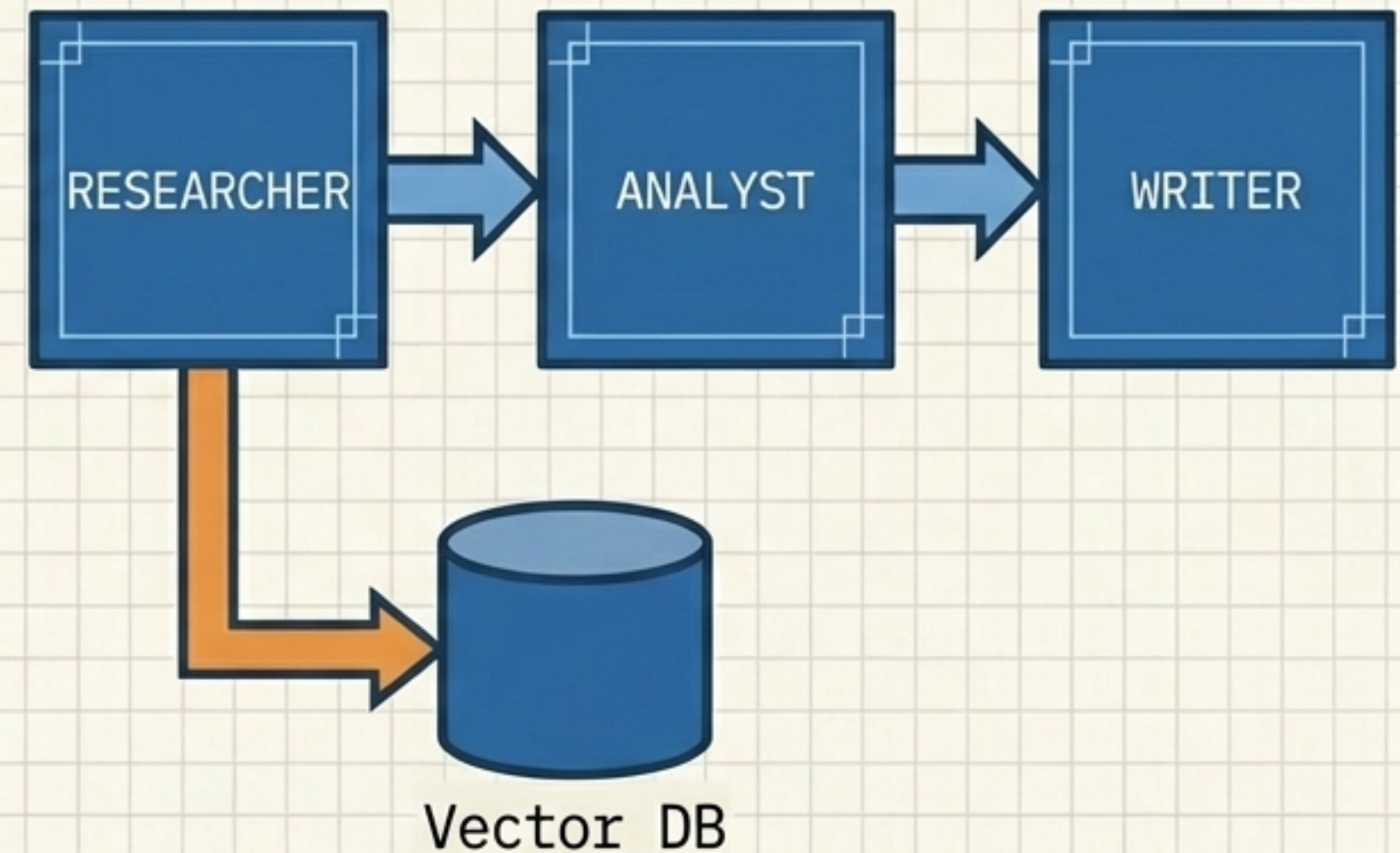
# Agent 03: Multi-Agent Routing via LangGraph

- **The Pattern:** A central Supervisor orchestrates multiple domain-expert agents.
- **Specialists:** Routes intent to Financial, Legal, Technical, or Summary domain agents.
- **Orchestration:** Supervisor decides dynamically if another expert is required.
- **Synthesis:** A dedicated Synthesizer agent combines insights into one cohesive response.



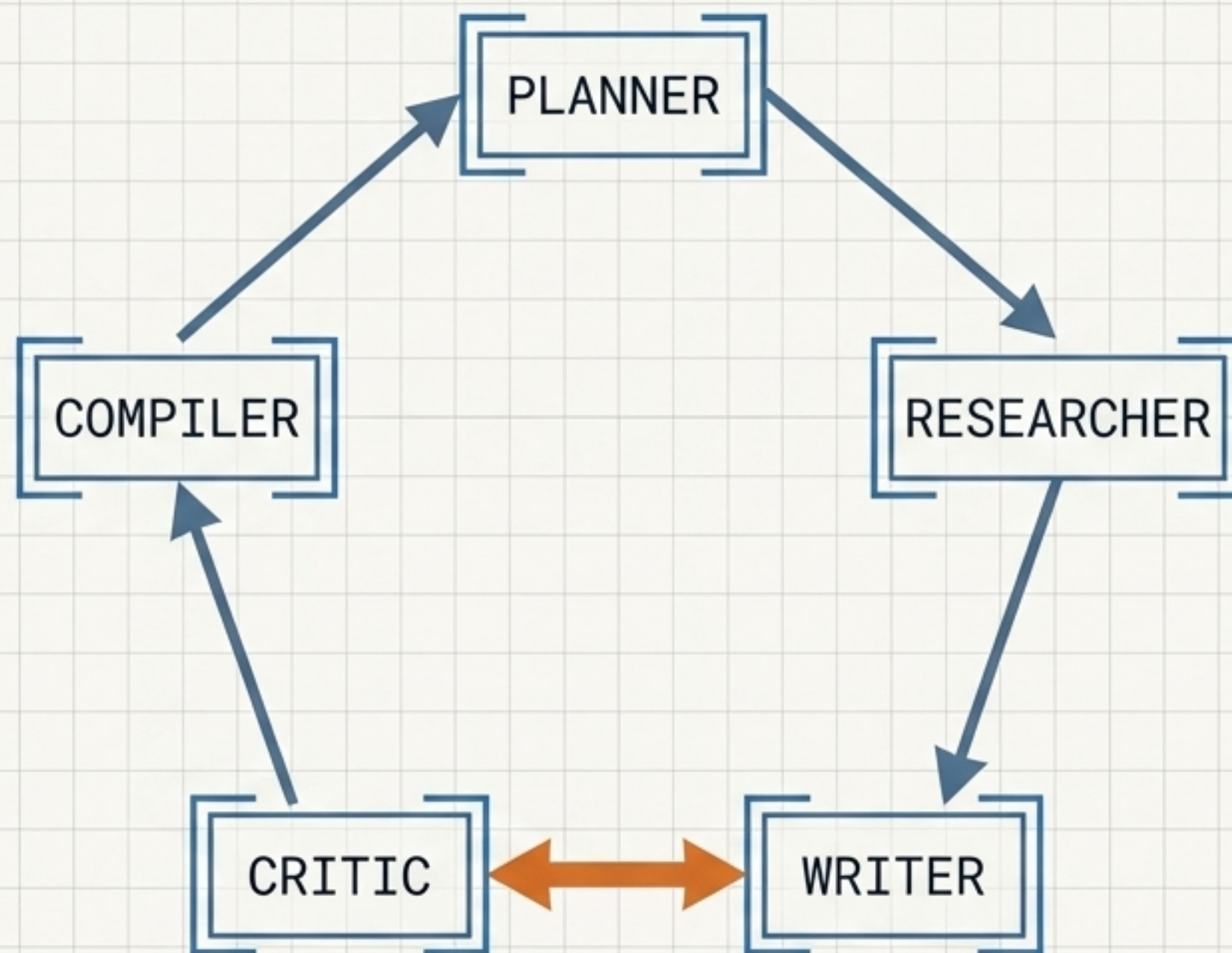
# Agents 04 & 05: CrewAI Role-Based Pipelines

- **Agent 04:** Uses abstraction (Role, Goal, Backstory) for highly contextual prompting.
- **Agent 05:** Creates a strict, sequential 3-stage assembly line workflow.
- **Stage 1 (Researcher):** Extracts raw facts with citations using vector DB tools.
- **Stage 2 (Analyst):** Interprets financial metrics and trends (no tool access).
- **Stage 3 (Writer):** Formats final executive-style response (no tool access).



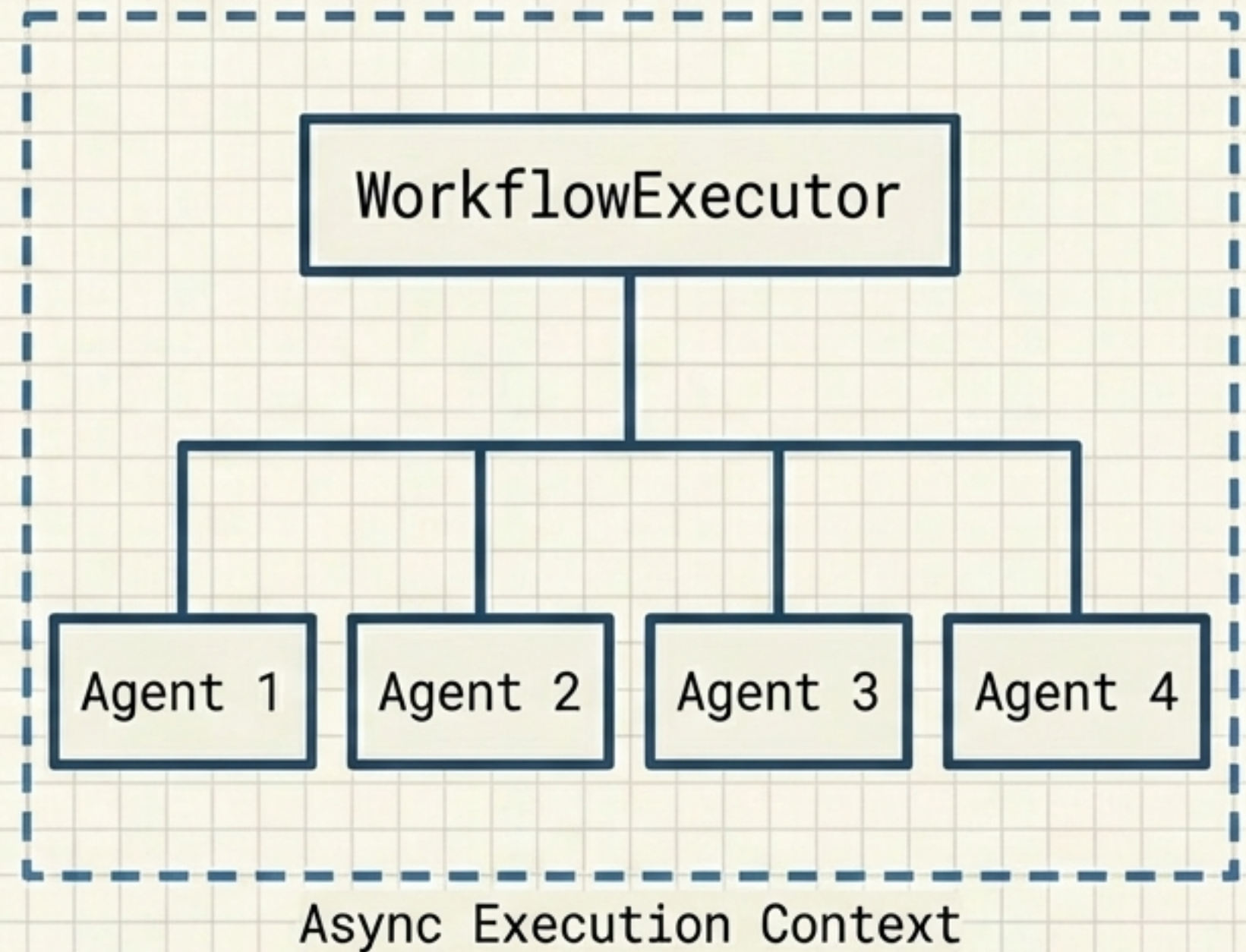
# Agent 06: AutoGen Collaborative Team

- **The Pattern:** A 5-agent team utilizing round-robin, turn-based conversation.
- **The Roster:** Planner, Researcher, Writer, Critic, and Compiler.
- **Peer-Review Loop:** The Critic actively reviews the Writer's draft for gaps before finalizing.
- **Autonomy:** System terminates when an agent outputs 'TERMINATE' or hits max rounds.



# Agents 07 & 08: Microsoft Agent Framework

- **The Pattern:** Native MS SDK applied to single and multi-agent RAG architectures.
- **Flexibility:** Utilizes an OpenAI-compatible client pointed at Google Gemini models.
- **Orchestration:** Recreates the Supervisor + Specialist pattern using WorkflowExecutor.
- **Execution:** Operates entirely within an async execution context for high concurrency.



# The Synthesis: Agent Framework Comparison Matrix

| FRAMEWORK    | PARADIGM              | BEST FOR                     | COMPLEXITY                               |
|--------------|-----------------------|------------------------------|------------------------------------------|
| LangGraph    | Custom Graphs / Nodes | Fine-grained routing control | High <input checked="" type="checkbox"/> |
| CrewAI       | Sequential Pipelines  | Predictable roleplay tasks   | Low <input type="checkbox"/>             |
| AutoGen      | Conversational Agents | Brainstorming & peer-review  | Medium <input type="checkbox"/>          |
| MS Framework | WorkflowExecutor      | Native enterprise ecosystems | Medium <input type="checkbox"/>          |

# The RAG Mastery Progression

- **Start with Infrastructure:** Solid ingestion, metadata, and APIs are prerequisites.
- **Retrieval is King:** MMR and Hybrid search solve 80% of accuracy issues.
- **Escalate Slowly:** Master ReAct basics before adding self-correction or supervisors.
- **Config Over Code:** Build resilient systems where models swap seamlessly via YAML.

